

Object-oriented programming in C++

Structs and pointers

Dr. Claudia Weis, Faculty of Computer Science

Fakultät für Informatik

UNIVERSITÄT DUISBURG ESSEN

Offen im Denken

Last week: Fundamental data types

- Integer Numbers : `…, -3, -2, -1, 0, 1, 2, 3, …`
- Floating Point Numbers: `0.5, 3.141, 42.0, …`
- Boolean: `true, false`
- Char: `'a'`
- String: `"Hello Class"`
- Array: `[1,2,3]`

- purpose: storing information in a computer and use them for computation, i.e. creating new information



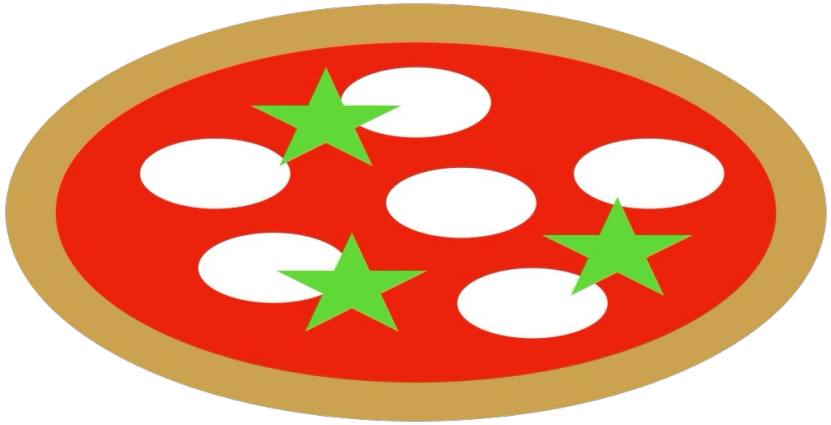
building our own data types

`struct Pizza`

Example scenario

- We are hungry and want to buy a price-efficient pizza.

- Offers
  - 29cm, 8.90 Euro
  - 25cm, 7.90 Euro
  - 30cm, 9.90 Euro



- Where do we get more pizza for our money?

How to enter several pizzas and compare their pricing?
- very simple solution -

Fakultät für Informatik

UDE

```
9   int main()
10  {
11      float pizza_1_price = 8.9;
12      float pizza_1_diameter = 29.0;
13      float pizza_2_price = 7.9;
14      float pizza_2_diameter = 25.0;
15      float pizza_3_price = 9.9;
16      float pizza_3_diameter = 30.0;
17
18      float cm_2_per_euro_pizza_1 = (pizza_1_diameter/2)*(pizza_1_diameter/2)*3.14159/pizza_1_price;
19      float cm_2_per_euro_pizza_2 = (pizza_2_diameter/2)*(pizza_2_diameter/2)*3.14159/pizza_2_price;
20      float cm_2_per_euro_pizza_3 = (pizza_3_diameter/2)*(pizza_3_diameter/2)*3.14159/pizza_3_price;
21
22      std::cout << cm_2_per_euro_pizza_1 << std::endl;
23      std::cout << cm_2_per_euro_pizza_2 << std::endl;
24      std::cout << cm_2_per_euro_pizza_3 << std::endl;
25  }
```

Phew,... Lots of copying/pasting...

Struct
Group variables, create concepts

- We often have data that semantically belong together.
  - e.g. price and diameter of our pizza
  - e.g. x and y coordinate of a point in 2d

- We want to make clear that they belong together.
- We want to simplify access/management.

- In C++: `struct`

Defining and creating a struct (1)
Example: point with x and y coordinate

Fakultät für Informatik

UDE

Name of the new "type"

```
struct Point {
  double x;
  double y;

  Point() : x(0), y(0) {}

  Point(double _x, double _y) : x(_x), y(_y) {}
};
```

member variables, i.e the "grouped together" variables

constructors (modern, efficient C++ style)

A constructor is used every time a new variable of type Point is created.
Here we have two of them: a default constructor that is called without parameters and assigns some default values to the member variables, and a second constructor with arguments in which the member variables are directly set to the values of the parameters

Defining and creating a struct (2)
Example: point with x and y coordinate

Fakultät für Informatik

UDE

```
struct Point {
  double x;
  double y;

  Point() {
      x = 0.0;
      y = 0.0;
  }

  Point(double _x, double _y) {
    x = _x;
    y = _y;
  }
};
```

older C++ style, still widely spread and maybe more appealing, especially to programmers who know Java or C#

Back to the Pizza example
- improvement #1 -

```
7   struct Pizza{
8       float price;
9       float diameter;
10  };
11
12  int main()
13  {
14      Pizza pizza_1 = {8.9, 29.0};
15      Pizza pizza_2 = {7.9, 25.0};
16      Pizza pizza_3 = {9.9, 30.0};
17
18      float cm_2_per_euro_pizza_1 = (pizza_1.diameter/2)*(pizza_1.diameter/2)*3.14159/pizza_1.price;
19      float cm_2_per_euro_pizza_2 = (pizza_2.diameter/2)*(pizza_2.diameter/2)*3.14159/pizza_2.price;
20      float cm_2_per_euro_pizza_3 = (pizza_3.diameter/2)*(pizza_3.diameter/2)*3.14159/pizza_3.price;
21
22      std::cout << cm_2_per_euro_pizza_1 << std::endl;
23      std::cout << cm_2_per_euro_pizza_2 << std::endl;
24      std::cout << cm_2_per_euro_pizza_3 << std::endl;
25  }
```

It isn't even necessary to write down the constructors, if it is obvious what they should be.
Note, however, the syntax with curly {}

Note here, how to address the member variables of a struct using the dot.

Pizza example
- improvement #2 -

- add a method to compute the amount of pizza per euro

```
7   struct Pizza{
8       float price;
9       float diameter;
10
11      float cm_2_per_euro(){
12          float radius = diameter/2;
13          return radius * radius * 3.14159 / price;
14      }
15  };
16
17  int main()
18  {
19      Pizza pizza_1 = {8.9, 29.0};
20      Pizza pizza_2 = {7.9, 25.0};
21      Pizza pizza_3 = {9.9, 30.0};
22
23      float cm_2_per_euro_pizza_1 = pizza_1.cm_2_per_euro();
24      float cm_2_per_euro_pizza_2 = pizza_2.cm_2_per_euro();
25      float cm_2_per_euro_pizza_3 = pizza_3.cm_2_per_euro();
26
27      std::cout << cm_2_per_euro_pizza_1 << std::endl;
28      std::cout << cm_2_per_euro_pizza_2 << std::endl;
29      std::cout << cm_2_per_euro_pizza_3 << std::endl;
30  }
```

Method

- A method is similar to a function (but strictly speaking not the same)
  - Written with the same syntax as a function, but inside a struct
  - Can access member variables directly
- Reduces code redundancy (“Don’t repeat yourself”, DRY Principle)
  - Write code once, reuse it multiple times
  - Easier to maintain: changes (or bug fixes) only needed in one place
- Encapsulate behaviour
  - Bundles data together with the functions that operate on it
  - Keeps object state consistent and protected

Method vs. function

Methods

- implemented as a part of a struct (or a class) to operate on objects
- implicit access to member variables of object

Functions

- exist independently and can be called directly
- access only to passed parameters, global objects

Pizza example – once again

```
7   struct Pizza{
8       float price;
9       float diameter;
10
11      float cm_2_per_euro(){
12          float radius = diameter/2;
13          return radius * radius * 3.14159 / price;
14      }
15  };
16
17  int main()
18  {
19      Pizza pizza_1 = {8.9, 29.0};
20      Pizza pizza_2 = {7.9, 25.0};
21      Pizza pizza_3 = {9.9, 30.0};
22
23      float cm_2_per_euro_pizza_1 = pizza_1.cm_2_per_euro();
24      float cm_2_per_euro_pizza_2 = pizza_2.cm_2_per_euro();
25      float cm_2_per_euro_pizza_3 = pizza_3.cm_2_per_euro();
26
27      std::cout << cm_2_per_euro_pizza_1 << std::endl;
28      std::cout << cm_2_per_euro_pizza_2 << std::endl;
29      std::cout << cm_2_per_euro_pizza_3 << std::endl;
30  }
```

We still have many lines of code that resemble each other.

→ Let's reduce the amount of "copy and paste".

Pizza example
- improvement #3 -

- keep objects of the same type together in an array
- iterate through the array

```
13  struct Pizza{
14      float price;
15      float diameter;
16
17      float cm_2_per_euro(){
18          float radius = diameter / 2;
19          return radius * radius * 3.14159 / price;
20      }
21  };
22
23  int main()
24  {
25      Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(int i = 0; i < 3; i++){
27          std::cout << pizzas[i].cm_2_per_euro() << std::endl;
28      }
29  }
```

Creating the array of objects of type pizza - a very careful, step-by-step look at it -

Fakultät für Informatik

UDE

number of elements that we want to store in the array (in squared brackets)

Each element (here of type `pizza`) is defined inside curly brackets

```
Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9, 30.0}};
```

name of the array (the programmer, i.e. we, can choose it)

We list all elements of the array inside curly brackets and separate them by commas.

type of each of the elements inside the array

Remember the assignment by `=` and the `;` at the end of the line ;-)

Pizza example, once again
- improvement #4 -

Fakultät für Informatik

UDE

- use modern style for the for-loop

old
"handmade"

```
23  int main()
24  {
25      Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(int i = 0; i < 3; i++){
27          std::cout << pizzas[i].cm_2_per_euro() << std::endl;
28      }
29  }
```

new
with automatic detection
of the object type
and the array length

```
23  int main()
24  {
25      Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(auto pizza: pizzas){
27          std::cout << pizza.cm_2_per_euro() << std::endl;
28      }
29  }
```

Foreach loop

- Iterates over elements in a collection
- Reduces probability of “off by one” errors
- `auto` for “automatic type inference”

```
25  Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(auto pizza: pizzas){
27          std::cout << pizza.cm_2_per_euro() << std::endl;
28      }
```

Before/after comparison

Fakultät für Informatik

UDE

```
9   int main()
10  {
11      float pizza_1_price = 8.9;
12      float pizza_1_diameter = 29.0;
13      float pizza_2_price = 7.9;
14      float pizza_2_diameter = 25.0;
15      float pizza_3_price = 9.9;
16      float pizza_3_diameter = 30.0;
17
18      float cm_2_per_euro_pizza_1 = (pizza_1
19      float cm_2_per_euro_pizza_2 = (pizza_2
20      float cm_2_per_euro_pizza_3 = (pizza_3
21
22      std::cout << cm_2_per_euro_pizza_1 <<
23      std::cout << cm_2_per_euro_pizza_2 <<
24      std::cout << cm_2_per_euro_pizza_3 <<
25  }
```

```
13  struct Pizza{
14      float price;
15      float diameter;
16
17      float cm_2_per_euro(){
18          float radius = diameter / 2;
19          return radius * radius * 3.14159 / price;
20      }
21  };
22
23  int main()
24  {
25      Pizza pizzas[3] = {{8.9, 29.0}, {7.9, 25.0}, {9.9,30.0}};
26      for(auto pizza: pizzas){
27          std::cout << pizza.cm_2_per_euro() << std::endl;
28      }
29  }
```

Summary: How to improve code?
- at the level we have after 3 weeks of this semester -

Fakultät für Informatik

UDE

- Use functions or methods for code you want to use repeatedly.
- Group variables that belong semantically together in structures.
- Implement their behaviour as methods of these structures.
- Use arrays if you have a fixed number of objects of the same type.
- Prefer automatic iteration instead of implementing the counter yourself.

➢ Whenever there is the need to copy code and change it only a tiny bit, think of possible improvements.

Robert C. Martin - “Uncle Bob”

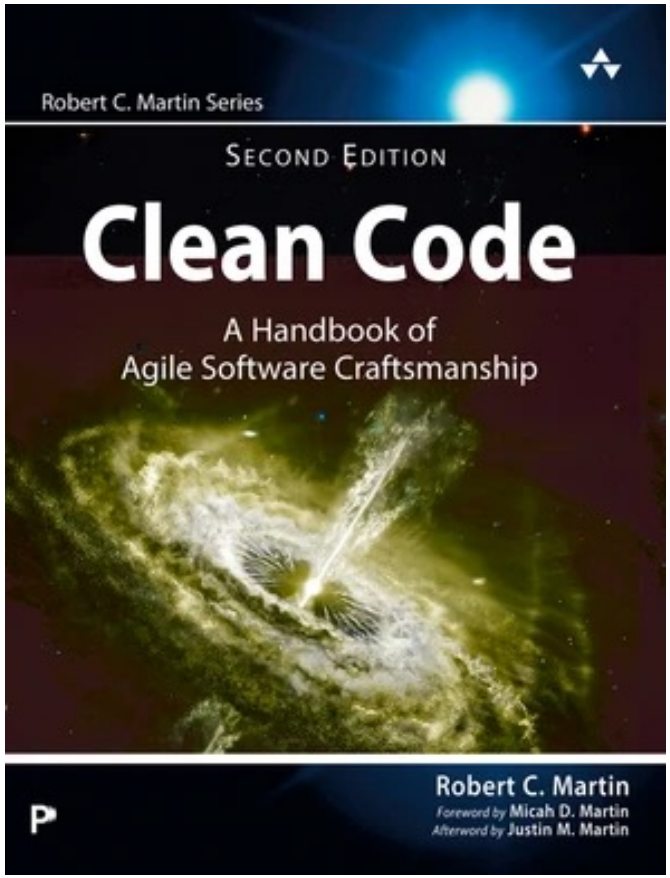
- Co-author of “Agile Manifesto” (2001)
  - foundations for how modern tech teams work today
- His Mission: "Software Craftsmanship"
  - coding is not just a factory job
- His philosophy: Write code in a way that the next developer (or you in 6 months) can understand without swearing. !
- However: Do not (yet) worry too much about details – Software Craftsmanship is a Master’s lecture at our faculty…
- We start with little -but important- steps



Further reading

- Clean Code: A Handbook of Agile Software Craftsmanship (2nd edition, 2026)

- available free of charge via UB
- accompanying video series



Fakultät für Informatik



Address Arithmetic

Pointers

What do variables store?

Fakultät für Informatik

UDE

`int`	→ stores an integer number (e.g., `42`)
`float`	→ stores a floating-point number (e.g., `3.14`)
`char`	→ stores a single character (e.g., `'h'`)
`string`	→ stores a piece of text (e.g., `"hello"`)
`pointer`	→ stores a memory address (which points to a location in memory)

Pointer: a variable that stores an address

- Pointers store addresses
  - to make live easier for us, C++ lets us keep the data type of what is stored at the address
- To get to the value, we need to look up what is stored at the address
- No need to be too scared of pointers ;-)



Why do we need pointers?
- one of many reasons -

Fakultät für Informatik

UDE

- Goal: Efficiently pass data to and from functions.
- Option A: Copy the data (“pass-by-value”)
  - Passing data creates full in-memory copies.
  - Issue: Significant runtime & memory overhead for large objects.
- Option B: Mutate in place (“pass-by-address”)
  - Pass the memory address (pointer) instead of copying the object.
  - Enables direct modification of the original object without copy overhead.
  - Trade-off / Risk: In-place mutation overwrites original data.
  - ➢ Requires explicit pointer management to prevent unintended side effects.

Control and flexibility with pointers

1. Fast & efficient access (pass-by-reference)
   - No memory overhead: Instead of copying large objects, we pass a lightweight memory address (pointer).
   - High performance: Function calls remain fast regardless of the object size.
2. Explicit in-place modification (when intended)
   - Direct mutability: Allows functions to modify the original object directly when needed.
   - Multiple outputs: Enables functions to return or update multiple values through their addresses.
3. Safety first: Read-only protection with `const`
   - Best of both worlds: Mark pointers as `const` to prevent accidental overwrites.
   - Guaranteed safety: Gives you the performance of pointers without sacrificing the safety of copies.

Example:
Fast read-only access without copying or risk of overwriting

Fakultät für Informatik

UDE

```
void processData(HeavyObject* data) {
    data->value = 42; // Changes original data
}



void processData(const HeavyObject* data) {
    // data->value = 42; // Compiler error if we un-comment:
                         // Protected from modification!
}
```

we'll talk about the -> operator in detail later

Side note

- When declaring a pointer both is possible

  ```
  int* a
  int *a
  ```

  - Both have the same meaning

- Recommendation for your own code:
  - Use `int* a`
  - this preserves the standard syntax of

    ```
    type name
    ```

  (here the name of the variable is a and its type is ”pointer to an integer”)

Warning:

Avoid declaring multiple variables/pointers on a single line!

- easily confusing:

  `int a, b` //declares two variables of type int

  `int *a, *b` //declares two pointer variables that point to an int

  `int* a, b` //declares one pointer to an int and an int variable (and NOT two pointers to int)

  `int *a, b` //declares one pointer to an int and an int variable

Pointers
Trivia and warnings

- A pointer technically needs no associated type
  - We *could* always use `void*`
  - Modern C++ does not allow it (for good reasons)
- Associated types are useful if we want to access and interpret the referenced value
- You can set a pointer to wrong locations and break your program



How to use pointers?
- easy example -

Fakultät für Informatik

UDE

C++ code

```cpp
int main() {

   // Declare integer variable
   int x = 42;

   // Pointer holds address of x
   int* p = &x;

   // Dereference 'p' to get value
   int val = *p;  // val == 42

   // Dereference to modify value
   *p = 99;       // x is now 99!

  return 0;

}
```

Key concepts

- & (Address-of Operator): Gets the memory address where a variable is stored (e.g., `&x -> 0x7ffd...`)
- * (Dereference Operator): Follows the memory address stored in a pointer to read or write the actual value.
- Reading (*p): Fetches the value stored at address p (returns 42).
- Writing (*p = 99): Overwrites the value at address p, directly modifying variable x.

Re-thinking pointers once more
- yet another example -

Address	Variable	Contains	Content
0x74	int x	Value	6
0x78	int* y	&x	0x74
0x82	int** z	&y	0x78
0x86	int*** a	&z	0x82

What happens and what is printed?

```cpp
#include <iostream>

  int main() {
      int x = 5;
      int* ptr = &x;

      x = 6;
      *ptr = 7;

      std::cout << x << std::endl;
      std::cout << ptr << std::endl;
      std::cout << *ptr << std::endl;
      return 0;
  }
```

```
> main.out
7
0x16b60f178
7

Process finished with exit code 0
```

What happens here?

Fakultät für Informatik

UDE

```
#include <iostream>

  int main() {


    int x = 7;
    int* ptr = 10;

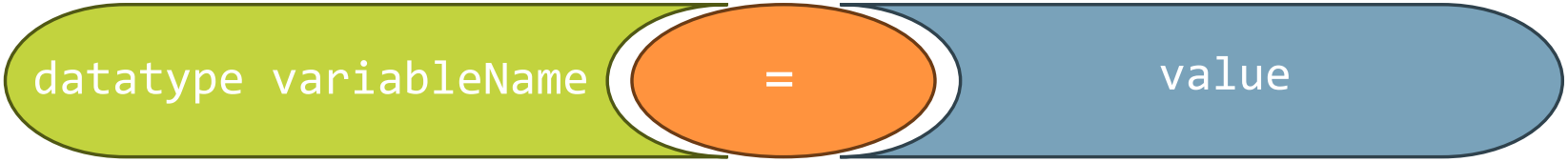

    std::cout << x << std::endl;
    std::cout << ptr << std::endl;
    std::cout << *ptr << std::endl;


    return 0;
}
```

Oups...

```
====================[ Build | untitled5 | Debug ]================================
/Applications/CLion.app/Contents/bin/cmake/mac/aarch64/bin/cmake --build
/Users/claudia/Uni/Lehre/OOP/untitled5/cmake-build-debug --target untitled5 -j 12
[1/2] Building CXX object CMakeFiles/untitled5.dir/main.cpp.o
FAILED: [code=1] CMakeFiles/untitled5.dir/main.cpp.o
/usr/bin/c++   -g -std=gnu++20 -arch arm64 -fcolor-diagnostics -MD -MT
CMakeFiles/untitled5.dir/main.cpp.o -MF CMakeFiles/untitled5.dir/main.cpp.o.d -o
CMakeFiles/untitled5.dir/main.cpp.o -c /Users/claudia/Uni/Lehre/OOP/untitled5/main.cpp
/Users/claudia/Uni/Lehre/OOP/untitled5/main.cpp:7:10: error: cannot initialize a
variable of type 'int *' with an rvalue of type 'int'
    7 |     int* ptr = 10;
      |          ^     ~
1 error generated.
ninja: build stopped: subcommand failed.
```

Assignment operator and type cast



- The data type of the value must match the data type set in the declaration.
- If the data type of the value is not identical to `datatype` in the declaration
  - sometimes implicit type cast possible
- type cast: conversion from one data type to another

Example

```
#include <iostream>

  int main() {

      float x = 5;
      int n = 5.5;
      bool b = 5.5;

      std::cout << x << std::endl;
      std::cout << n << std::endl;
      std::cout << b << std::endl;

      return 0;
  }
```

```
> main.out
5
5
1

Process finished with exit code 0
```

Another example

```
#include <iostream>

  int main() {

      char c = 42.4;


      std::cout << c << std::endl;


      return 0;
  }
```

```
> main.out
*

Process finished with exit code 0
```

Type cast “by hand”

```cpp
#include <iostream>
int main() {

  double pi = 3.14159;

  // modern C++ style (with some implicit safety)
  int wholeNumber1 = static_cast<int>(pi);

  // older C-like style (still widely used)
  int wholeNumber2 = (int)pi;

  return 0;
}
```

Explicit type casting tells the compiler
that the type cast is on purpose
Maybe the compiler gives a warning.
The programmer (you) must take the responsibly.

```
int* ptr = 10; // revisited
```

- What happens:
  - 10 is actually an integer
  - int* ptr needs an address
  - a modern C++ compiler gives an error, because an int is not an address and thus the assignment does not make sense

Forcing the type cast

```
#include <iostream>

  int main() {

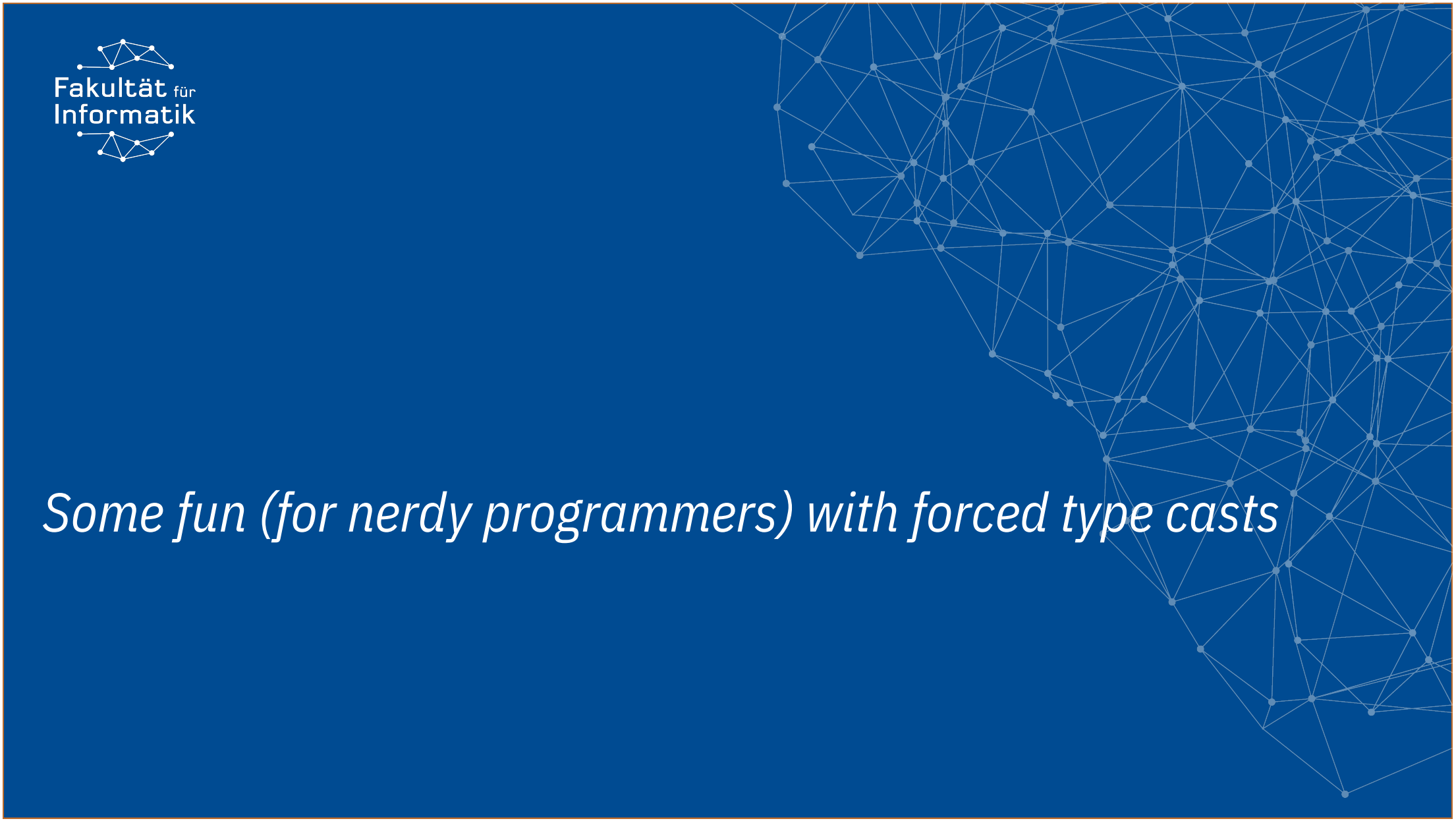
  int* ptr = (int*)10; //force the type cast using C style syntax
  std::cout << ptr << std::endl;
  std::cout << *ptr << std::endl;


  return 0;
}
```

```
> main.out
ptr: 0xa

Process finished with exit code 139 (interrupted by signal 11:SIGSEGV)
```

⟶ Program compiles but crashes when it tries to use *ptr



Some fun (for nerdy programmers) with forced type casts

Getting started – easy ;-)

```cpp
#include <iostream>

int main() {

    float minuspi = -3.14159;

    float* ptr1 = &minuspi;

    std::cout << "address where ptr1 points to: "  <<  ptr1 << std::endl;
    std::cout << "Let's see what is there: "       << *ptr1 << std::endl;

}
```

```
> main.out
address where ptr1 points to: 0x16f4ab17c
Let's see what is there: -3.14159

Process finished with exit code 0
```

The address where ptr1 points to can change in every single run of our little programme ;-)

How much memory does `float` *use?*

Fakultät für Informatik

UDE

```cpp
#include <iostream>

int main() {

   float minuspi = -3.14159;

   int size = sizeof(minuspi);

   std::cout << "Memory for minuspi (in byte): " << size << std::endl;
}
```

```
> main.out
Memory for minuspi (in byte): 4

Process finished with exit code 0
```

How many bit are 4 byte?

Lets look at the same address
- but with different interpretation of the value -

Fakultät für Informatik

UDE

```
#include <iostream>
#include <cstdint>

  int main() {

   float minuspi = -3.14159;
   float* ptr1 = &minuspi;

   int32_t* ptr2 = (int32_t*)&minuspi;

   std::cout << "Let's see what is there: "  << *ptr1 << std::endl;
   std::cout << "after type cast to int32_t: "  << *ptr2 << std::endl;
}
```

```
> main.out
address where ptr1 points to: 0x16dddf17c
Let's see what is there: -3.14159
address where ptr2 points to: 0x16dddf17c
after explicit type cast to int32_t: -1068953648

Process finished with exit code 0
```

A few more...

```cpp
#include <iostream>
#include <cstdint>

int main() {

  float minuspi = -3.14159;

  float* ptr1 = &minuspi;
  int32_t* ptr2 = (int32_t*)&minuspi;
  uint32_t * ptr3 = (uint32_t *)&minuspi;
  uint16_t * ptr4 = (uint16_t *)&minuspi;
  uint64_t * ptr5 = (uint64_t *)&minuspi;

  std::cout << "Let's see what is there: " << *ptr1 << std::endl;
  std::cout << "type cast to  int32_t: "   << *ptr2 << std::endl;
  std::cout << "type cast to uint32_t: "   << *ptr3 << std::endl;
  std::cout << "type cast to uint16_t: "   << *ptr4 << std::endl;
  std::cout << "type cast to uint64_t: "   << *ptr5 << std::endl;
}
```

```
> main.out
Let's see what is there: -3.14159
type cast to int32_t: -1068953648
type cast to uint32_t: 3226013648
type cast to uint16_t: 4048
type cast to uint64_t: 7705157382196236240
```

Remember: float is 32 bit. In the last case bits outside our original value are interpreted.

```
Process finished with exit code 0
```

The fun part ;-)

```
#include <iostream>
#include <cstdint>


int main() {
  float minuspi = -3.14159;
  uint32_t* ptr = (uint32_t *)&minuspi;

  for (int k = 31; k >= 0; k--)
      std::cout << ((*ptr >> k) & 1);
  }


  std::cout << std::endl;
}
```

```
> main.out
11000000010010010000111111010000

Process finished with exit code 0
```

Getting the details (1)

```
float minuspi = -3.14159;
uint32_t* ptr = (uint32_t *)&minuspi;
```

- Using `&minuspi` we get the address where `minuspi` is stored.

- What do we find when looking at this address into the memory?
- Starting at that address and taking the next 32 bits, we find the binary encoded value of our variable `minuspi` (which is -3.14159)

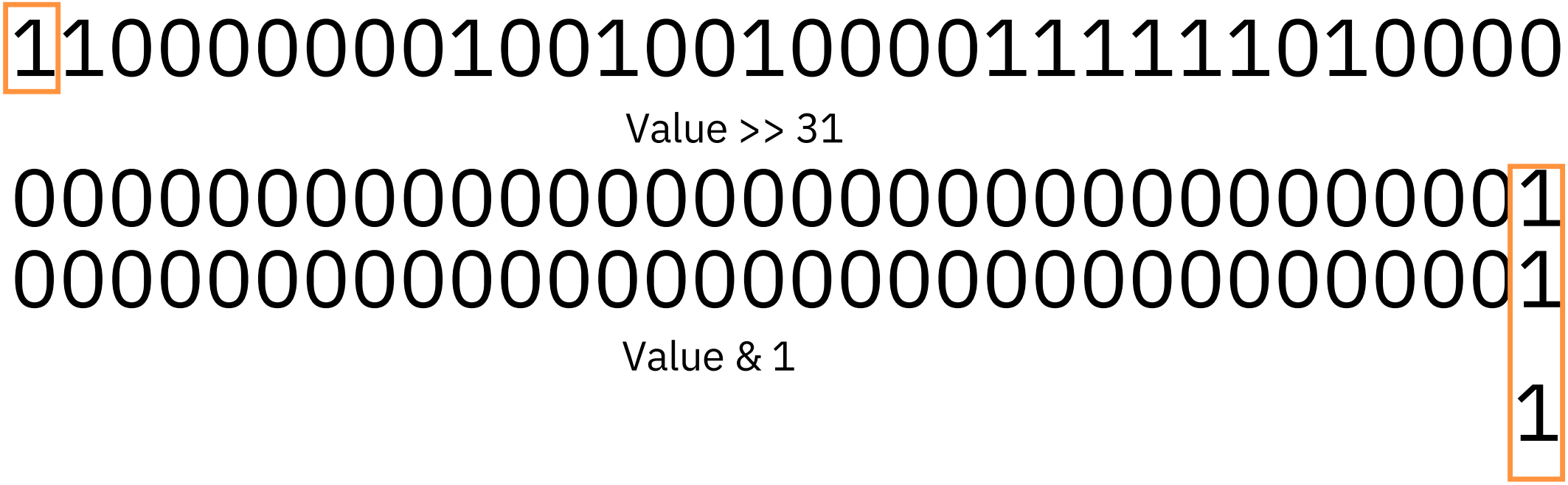
Getting the details (2)

```
for (int k = 31; k >= 0; k--)
      std::cout << ((*ptr >> k) & 1);
 }
```

- The for-loop iterates over 32 bits, starting from the most significant

`*ptr`	Reads the value from where ptr points to.
`*ptr >> k`	Shifts the value bitwise to the right by k positions which moves the kth bit to the right most position.
`((...) & 1)`	A bitwise AND (&) with 1 (binary ...0001) effectively strips away all other bits and clears them to 0.

- The result is simply 0 or 1, reflecting the state of the k-th bit which is then printed out.


11000000010010010000111111010000
Value >> 31
00000000000000000000000000000001
00000000000000000000000000000001
Value & 1
1



11000000010010010000111111010000

Value >> 30

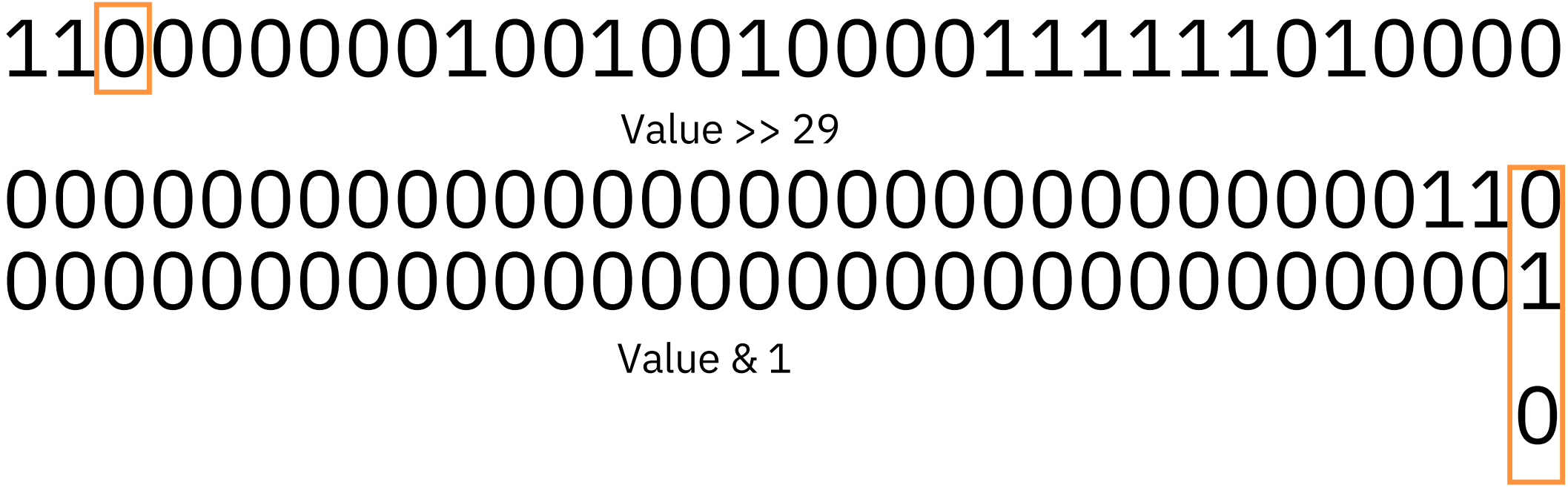
00000000000000000000000000000011

Value & 1



00000000000000000000000000000001

1


11000000010010010000111111010000
Value >> 29
00000000000000000000000000000110
00000000000000000000000000000001
Value & 1
0



1100000010010010000111111010000

Value >> 28

0000000000000000000000001100

0000000000000000000000000001

Value & 1

0



11000000010010010000111111010000

Value >> 27

00000000000000000000000011000

00000000000000000000000000001

Value & 1

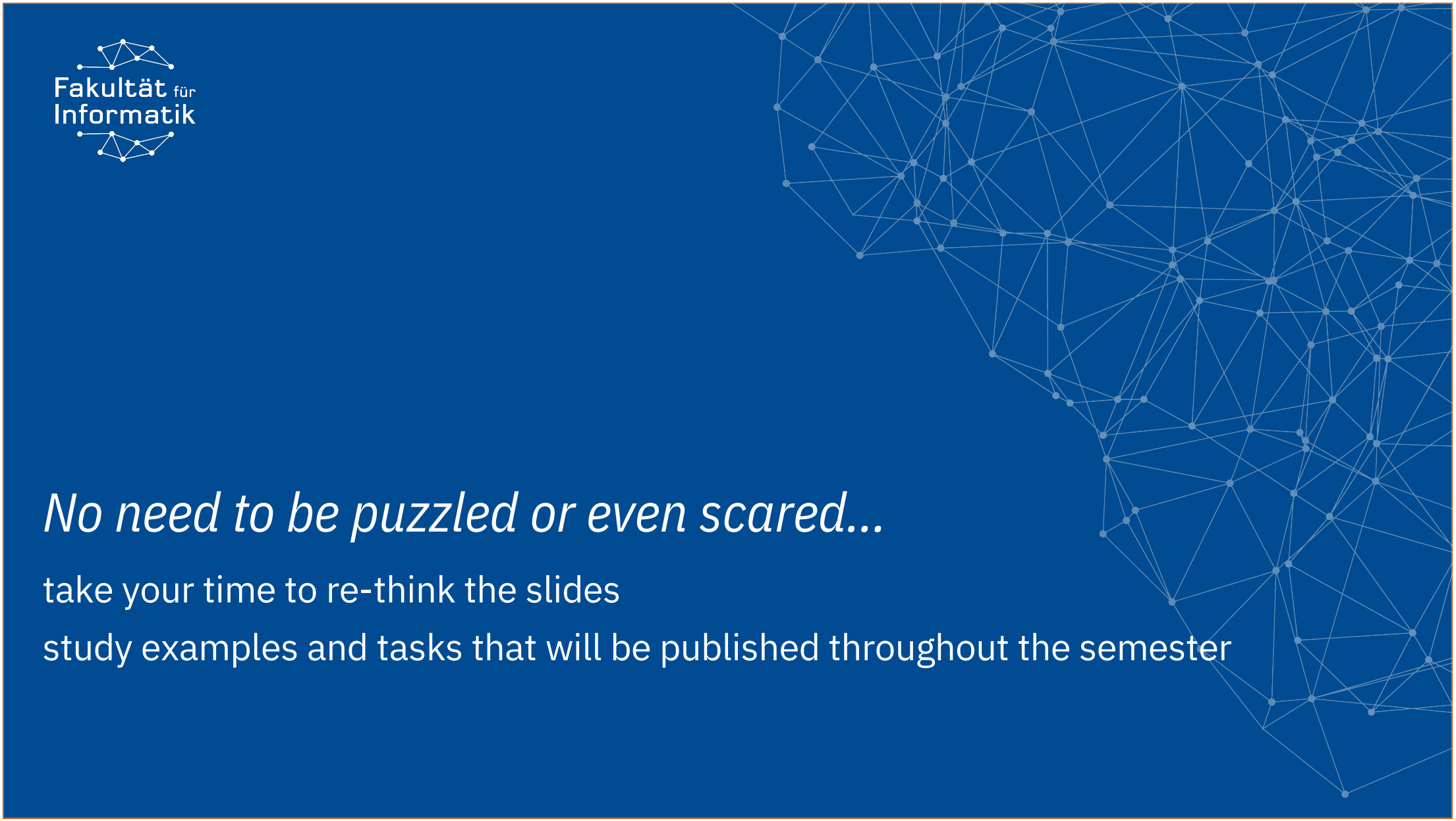
0

Getting the details (3)

Fakultät für Informatik

UDE

We printed the binary representation of a float (with negative sign), i.e. -3.14159 :-)



No need to be puzzled or even scared...

take your time to re-think the slides

study examples and tasks that will be published throughout the semester

Remember? Why did we start to talk about pointers?



Pointer vs. Reference
same but different – the modern C++ solution

Fakultät für Informatik

UDE

```
7   void add_1_value(int value){
8       value += 1;
9   }
10
11  void add_1_pointer(int* value){
12      *value += 1;
13  }
14
15  void add_1_reference(int& value){
16      value += 1;
17  }
```

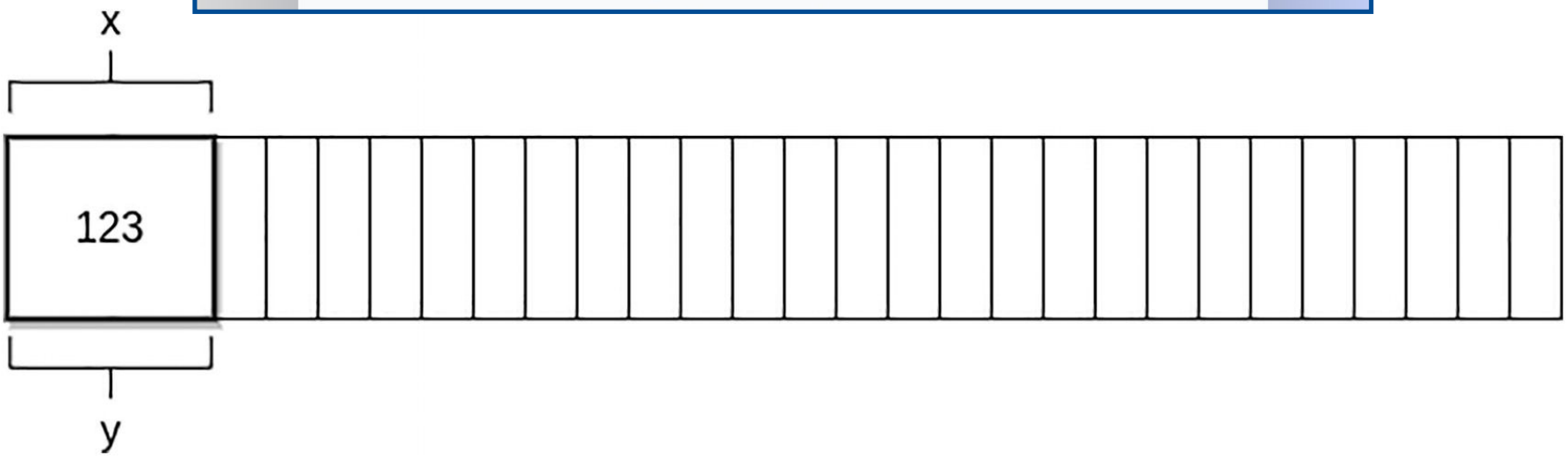
```
18
19  int main()
20  {
21      int x = 5;
22      add_1_value(x);
23      std::cout << x << std::endl;
24
25      add_1_pointer(&x);
26      std::cout << x << std::endl;
27
28      add_1_reference(x);
29      std::cout << x << std::endl;
30  }
```

```
❯ ./a.out
5
6
7
```

Reference: alias and automatic dereference

- A reference is a nice feature to simplify working with addresses instead of values.
- It is another variable that refers to the same memory location

```
35        int x = 123;
36        int& y = x;
```



Example (in pseudo code)

Let's assume the following scenario. How could we implement it?

```
struct DataHolder {
     int x;
}
```

```
DataHolder addOne( parameter_of_type DataHolder )
     add plus 1
}
```

```
int main() {
     // (1) create variable of type DataHolder
     // (2) add plus 1 to the x of our previously
     // created variable using the function addOne
}
```

Solution 1: Call-by-value

```
6   struct DataHolder {
7       int x;
8   };
9
10  DataHolder addOne(DataHolder dataHolder){
11      dataHolder.x ++;
12      return dataHolder;
13  }
14
15  int main(){
16      DataHolder x = {1};
17      x = addOne(x);
18  }
```

What happens?
line 16: x is created
line 17: x is handed over to the function whereby a copy of the original object is used.
The function operates on the copy and returns value+1 where the new value is assigned to x.
A lot of values are copied or written newly to the memory.

Solution 2: Use pointer

```
6    struct DataHolder {
7        int x;
8    };
9
10   void addOne(DataHolder* dataHolder){
11       (*dataHolder).x++;
12   }
13
14   int main(){
15       DataHolder x = {1};
16       addOne(&x);
17   }
```

What happens?
line 15: x is created
line 16: The address of x is handed over to the function.
The function operates on the original and changes the value at the original place.
Do you understand why we have *, & and . (dot) at the different places?
Where do we have to use additional ()?

Solution 2b: Use pointer but with "nicer" syntax

```
6   struct DataHolder {
7       int x;
8   };
9
10  void addOne(DataHolder* dataHolder){
11      dataHolder->x++;
12  }
13
14  int main(){
15      DataHolder x = {1};
16      addOne(&x);
17  }
```

What happens?
line 15: x is created
line 16: The address of x is handed over to the function.
The function operates on the original and changes the value at the original place.
Do you understand why we have *, & and . (dot) at the different places?
Where do we have to use additional ()?

Solution 3: Call-by-reference

```
6    struct DataHolder {
7        int x;
8    };
9
10   void addOne(DataHolder& dataHolder){
11       dataHolder.x++;
12   }
13
14   int main(){
15       DataHolder x = {1};
16       addOne(x);
17   }
```

What happens?
line 15: x is created
line 16: x is handed over to the function.
Due to the function definition, a reference to x is used and the function operates on the original object.
We only have to care about, what the function definition looks like to know, if your original object in main() is changed.

This is modern C++ style.

Summary

Fakultät für Informatik

UDE

- We introduced user-defined data types: struct
- We solved the problem of efficiently passing large objects to functions.
- Besides we learned a lot about using pointers.

Operators – some more considerations

- Infix operator



- algorithm how to calculate depends on data types of left and right operands
- Example: multiplication operator ·
  - integer · integer = integer
  - float · float = float
  - row vector · column vector = number (dot product, if dimensions match)
  - matrix · column vector = row vector (if dimensions match)

Operator Overloading
- Defining operators on our own datatypes -

- Aim: give standard operators (+, -, *, ==) custom meanings for user-defined types
- Purpose: makes custom objects behave intuitively like built-in types (int, float).
  - “syntactic sugar”
- Timing: Evaluated at compile-time (static).

- Example:
  - Let v1 and v2 two vectors.
  - We’d like to write `"v1+v2"` instead of something like `"v1.add(v2)"`

Example 1: our own data type for complex numbers

- two member variables: real and imaginary part
- operator overloading for +

- Exercise: extend our datatype with more operators and a printing function that prints a complex number in the typical format `real + i imag`

```
3   struct Complex {
4       int real, imag;
5
6       Complex(int r = 0, int i = 0){
7           real = r;
8           imag = i;
9       }
10
11      Complex operator+(Complex const& obj)
12      {
13          Complex res;
14          res.real = real + obj.real;
15          res.imag = imag + obj.imag;
16          return res;
17      }
18  };
19
20  int main()
21  {
22      Complex c1(10, 5), c2(2, 4);
23      Complex c3 = c1 + c2;
24  }
```

Example 2: our own data type for time

```
3   struct Time {
4       int _hour;
5       int _minute;
6       int _second;
7
8       Time() : _hour(0), _minute(0), _second(0) {}
9       Time(int h, int m, int s) : _hour(h), _minute(m), _second(s) {}
10
11      int const hour() const {return _hour;}
12      int const minute() const {return _minute;}
13      int const second() const {return _second;}
```

3 member variables

constructors

3 methods

Example 2 cont'd: We'd like to add times using +

```
70  int main() {
71      Time t1(12,23,50);
72      Time t2 (1,1,1);
73      Time t3 (1,50,20);
74
75      std::cout << (t1 + t2) << std::endl;
76
77      std::cout << (t1 + t3) << std::endl;
78      return 0;
79  }
```

Overloading the + operator
- very simple version -

```
15    Time operator+(const Time& rhs) const {
16          Time t(_hour + rhs._hour, _minute + rhs._minute, _second + rhs._second);
17          return t;
18       }
```

define the operator overloading similar to a method
(within the definition of our struct Time)

☐ Unfortunately, our super simple implementation here suffers from a typical problem.
Who sees it?

What if your time overflows?



Helper method: `sanitize()`

```
21      void sanitize() {
22          while (_second > 59) {
23              _minute += 1;
24              _second -= 60;
25          }
26          while (_minute > 59) {
27              _hour += 1;
28              _minute -= 60;
29          }
30      }
```

We can use it for other (future) operations as well.

Overloading the + operator
- updated version -

parameter by reference,
const to forbid modification of the time we want to add

another const here, to forbid
modification of the object itself

```
15      Time operator+(const Time& rhs) const {
16          Time t(_hour + rhs._hour, _minute + rhs._minute, _second + rhs._second);
17          t.sanitize();
18          return t;
19      }
```

return by value
we anyway have to create a new object (that needs new memory)
holding the sum when we add two objects of type Time

Overloading the += operator

```
33      Time& operator+=(const Time& rhs) {
34          _hour += rhs._hour;
35          _minute += rhs._minute;
36          _second += rhs._second;
37          sanitize();
38          return *this;
39      }
```

details on next slide

return by reference
we change the original object and thus returning a reference

What is `*this`?
(in C++)

- implicit pointer that exists within every non-static member function (method) of a struct (or a class).
- It points directly to the memory address of the object that invoked the method.
- When you write `*this`, you dereference that pointer and get the object itself as a reference.

```
return *this;
```

- returns the current object from within a member function.

Options to write things down (1)

```
32      myDate.setMonth( 3 );
33      setMonth( &myDate, 3 );
```

the two lines are semantically the same
(assuming that setMonth is a method for the datatype of myDate)

Options to write things down (2)

```
32  void Date::setMonth( int mn )
33  {
34     month = mn;             // These three statements
35     this->month = mn;       // are equivalent
36     (*this).month = mn;
37  }
```

(assuming that setMonth is a method within the datatype Date)

Homework

- Assume Time is a user-defined data type as in our previous example.

- Why does it make sense to return `Time&` when overloading += and `Time` when overloading +?

- Explain the differences between
  a) `Time&` myFunction(someParameters)
  b) `Time` myFunction(someParameters)

Fakultät für Informatik

Questions?



"live" in lecture and exercise



discussion forum @ Moodle (preferred, everyone can reply)



by e-mail (use your @uni-due.de email address!)